



Modeling users' preference changes in recommender systems via time-dependent Markov random fields

Abinash Pujahari ^{a,b,*}, Dilip Singh Sisodia ^a

^a Department of Computer Science & Engineering, National Institute of Technology Raipur, GE Road, Raipur, Chhattisgarh 492010, India

^b Department of Computer Science Engineering, SRM University, Amaravati 522502, AP, India

ARTICLE INFO

Keywords:

Dynamic preferences
Collaborative filtering
Pair-wise preferences

ABSTRACT

Recommender Systems are helpful to many by filtering the information according to an individual's preferences. However, the choice of a person may change with time. Keeping track of these changes or predicting the next sequence of items is difficult for any recommender system. Most time-aware recommender systems exploit users' rating distribution to predict the next possible set of items. But due to the sparsity (users' preferences) in real-world datasets, these models suffer from low accuracy. To overcome such issues, this study proposes a collaborative filtering model which uses a preference estimation technique that uses user and item feature information along with the ratings. Further, we have used preference relations (instead of ratings) to improve the ranking quality of the recommender system. We have used the ML-1M, NetFlix, and Last.fm datasets for evaluation, the benchmark datasets for recommender systems testing. All these three datasets have sparsity levels above 92%. Experimental results indicate significant improvement (5 to 7%) over the state-of-the-art related baselines.

1. Introduction

Capturing users' preference changes is a difficult task for any Recommender System (RS) (Milano et al., 2020; Wang et al., 2021; Wu, He, et al., 2022). Most Collaborative Filtering (CF) models (Koren et al., 2021; Pujahari & Sisodia, 2020a; Wu, Sun, et al., 2022) (used for generating recommendations) rely on predicting the missing ratings and ignore the users' preference change. Some temporal RS models (Ait Hammou et al., 2019; Pujahari & Sisodia, 2022; Yang et al., 2022) also consider rating distributions over a period of time to capture users' behaviors towards different items. Several factors induce changes in users' tastes, like age, demographic change, etc. A pleasing RS should adopt such situations (Gharahighehi & Vens, 2023; Panagiotakis et al., 2021; Zhao et al., 2022) to provide more accurate recommendations and keep the end users engaged in increasing the productivity/performance of any organization.

There are some CF models proposed recently for capturing users' preference change. Some of them (Ahmadian et al., 2022; Narang et al., 2021; Zhang et al., 2022) follows the latent feature approach, where the user-item interactions are predicted. These models either use some kind of weighting strategy (using a time variable) during the learning of the latent feature matrices. Some of them also use the concept of neural network or graph neural network (Jalali & Hosseini, 2022; Zhang et al., 2022) for learning the user item latent features. Then there are

some dynamic recommender system models (Wang et al., 2022; Zhang et al., 2021) that uses tensor factorization approach. These models use one additional matrix, i.e. time matrix in addition to user and item latent feature matrices. All these three matrices are combined to predict the missing ratings in a user-item rating matrix. We have some probabilistic CF models, that studies the ratings distribution (Hazrati & Ricci, 2022) of users over certain time periods (based on the density of the datasets) to predict the next possible sequence of items that the user might like.

All the techniques involved in dynamic recommendation (or temporal recommendations) involved in learning latent feature (Lo et al., 2018; Rafailidis & Nanopoulos, 2016) or distribution estimation (Sahoo et al., 2012; Zhang et al., 2018) for predicting the next possible sequence of items to be consumed by users. For example, the user-item interactions (e.g. ratings), will be partitioned with fixed time intervals (e.g., six months) (Pujahari & Sisodia, 2022). The ratings of the previous six months (or any duration defined by the model) will be analyzed to predict the next probable list of items (Jalali & Hosseini, 2022) that the user will consume. In these techniques, the user-item interactions along with their time stamp is required. Most of these models cannot justify (Lin et al., 2022) the recommendation because they do not use the feature information (Carroll et al., 2022). Generally,

* Corresponding author at: Department of Computer Science Engineering, SRM University, Amaravati 522502, AP, India.

E-mail addresses: abinashpujahari@ieee.org (A. Pujahari), dssisodia.cs@nitrr.ac.in (D.S. Sisodia).

the users tend to change their taste by comparing the features of different items. Hence, mere analyzing the predicted ratings or distribution is not enough for generating time-aware recommendations. It is very difficult to integrate the item feature information (Sun et al., 2022) in latent feature based CF models. Also good prediction of ratings do not ensure good recommendation (Patro et al., 2022) because a good recommendation always puts the highest preferred items at the top of the list. Most of the models puts the highest rated items at the top of the recommendation list during recommendations.

Considering the different techniques discussed above to capture the change in users' preferences in RSs, it can be observed that the latent feature based models are still not fully capable of modeling dynamic user behaviors. The reason behind this is that the latent factor models are capable of reducing sparsity in the dataset. For modeling preference change we need to change the technique that is capable of holding the transition in users' preferences. Some of the models focused on this research using techniques like *Hidden Markov Model* (Sahoo et al., 2012; Zhang et al., 2018) etc. However these models use distribution of ratings to provide recommendation and may provide poor performance when they are provided with sparse datasets. Hence, this study provides a unique approach to model change in users preference over time which is summarized below:

- The proposed approach unifies the idea of point estimation (i.e., rating prediction) and distribution estimation (i.e., users' rating distribution over time).
- The latent feature part will address the sparsity issue in CF while the distribution estimation will model the preference transition, which is achieved through *Markov Random Fields* (MRF).
- Using MRF, we will be able to integrate the user and item features which is very crucial for determining what is the reason behind the preference change.
- The use of preference relations instead of ratings provides better ranking of items which is also crucial for any RS.

The further study has been organized as follows. We have given the analysis of the existing time-dependent RS models in Section 2. The proposed method is described in Section 3. Section 4 contains the experimental evaluation of this study. We have added Section 5 to discuss the outcome and advantage of the proposed model. The conclusion section aggregates the outcome along with a direction of future research in this study.

2. Background and recent works

We will cover this section in two parts. The first part describes the latent state-of-the-art time aware or dynamic users' preference RS models. We will also discuss their advantages and limitations and the motivation behind our proposed work. The second part describes the *Preference Relation*-based RS models and their advantage towards generating better ranking of items.

2.1. Time-dependent RS models

In this section we will describe about only those RS models, that uses transitive latent variable as well as time dependent latent factors for capturing change in users' preferences with respect to time. Hence we will describe the background of the proposed research in two different categories.

2.1.1. Latent-factor based models

In one of the recent latent-factor approach for temporal CF model (RSTRC) is proposed by Ahmadian et al. (2022). Here the authors proposed a unique user profile technique. The user profile is built using various steps like calculating temporal similarity values, temporal implicit rating values which defines the relative neighborhoodness among

different users. The significant contribution of this approach is that it considers the similarity in preference of different users which covers the basic requirement of neighborhood based recommendation. Further the use of temporal reliability measure, achieved through a weighted function makes the recommendation more robust. The authors modified the baseline predictor equation mentioned by Koren (2009) using the weighted user rating profile. The limitation of the model is that it does not use any feature information which may not perform well in case of extreme sparseness in data.

Another significant contribution to time-aware recommendation proposed by Zhao et al. (2022), i.e. *TPRec*, where the authors make use of knowledge graph for representing temporal features of user-item interactions. The major advantage of this approach is that it uses side information which is also used in our proposed model. The authors make use of the item features to be integrated to the knowledge graph model for latent feature generation. The temporal structure has been designed similar to our proposed model, which uses one probabilistic model based on the timestamp given in the dataset. We have used this model as one of the baseline during comparison. The temporal recommendation of this model is good but with a limitation that it do not consider the relative neighborhoodness among items and users.

Wang et al. (2022) proposed an approach (*TADCF*) to handle change in preference using deep learning frame work which focused on users' attentions. In addition to the normal deep learning framework for generating user and item latent factors, the authors used additional latent feature vector like long-term and short-term user preferences, time embedding vector. The time embedding vector is represented through a weighted matrix which is called the item encoding matrix. The dynamic user preference modeling involves in unification of all the weighted matrix discussed above. The prediction of ratings of unseen item is done using the dot product of the latent feature matrices. The temporal recommendation is good for this method but its application is limited to specific recommendation scenario and cannot be generalized to others.

Another approach to handle change in users' preference using latent factor model is proposed by Zhang et al. (2021). Here the authors used tensor factorization based model, named as *DTRS*. Here the authors used a function of time (tensor valued) for modeling users' preference which also considers the contextual information. This creates a time varying coefficient through polynomial spine approximation. This is a robust model for collaborative filtering when we have rich feature information available. The model proposed by Narang et al. (2021) named as *FuseRec* which combines all the information related to users preferences such as past behavior, his/her social interactions, and the similarity among items. The authors use embedding technique to merge all these information in a co-factorization framework. They validated their work using *Epinions* and *CiaoDVD* datasets. The temporal module is implemented using RNN. Most of these models focus on predicting ratings information and do not give emphasis on ranking of items. Also the neighborhood information is not considered in these models, which is the essence of CF.

2.1.2. Probabilistic/stochastic models

A significant contribution to temporal CF model(NHMM) proposed by Carroll et al. (2022) related to change in users' preferences, where they proposed a technique to estimate the preference shift and at the same time validate the same also. During training of the system the authors followed the Non-Homogeneous HMM (Hidden Markov Model) for the rating distribution estimation. Harshvardhan et al. (2022) proposed a time-aware recommendation model using Boltzmann machine. The learning of the method is done using the contrastive divergence procedure.

One of the significant contribution to dynamic user preference based RS model is proposed by Sahoo et al. (2012). In this model, the authors used *Hidden Markov Model* (HMM) to model change in preferences of users with fixed time interval. They projected users preferences as

a hidden Markov sequence. They have also used negative binomial mixture of multinomial to model the items selected by each user for each time interval. They have used fixed time interval for studying users preference changes. Recently, Zhang et al. (2018) extended this model using *Hidden Semi-Markov Model*, that captures the heterogeneity in duration of users' preference changes. The core idea of this model is to keep track of various duration of user stays in a state (i.e. preferences/latent). This model allows user to stay at different states for different time interval as compared to HMM model (where time interval is fixed). The performance of these two models are good when the dataset is having less sparsity but not so good when the dataset is too sparse.

2.2. Preference relation

Preference Relation (PR) is often discussed previously in *information retrieval* procedures. The first PR-based RS model was introduced by Brun et al. (2010), which was a neighborhood based (KNN) approach. Later, it was extended to Matrix Factorization (MF) framework by Desarkar et al. (2012). PR-based MF models outperformed the generic MF procedures for RSs (Desarkar et al., 2012; Pujahari & Sisodia, 2020b). Recently, PR was used in more RS models using Markov Random Fields (Liu et al., 2017), Restricted Boltzmann Machine (Pujahari & Sisodia, 2019), etc. As defined by Brun et al. (2010), PR for user u for an item pair (i, j) :

$$\pi(i, j)_u = \begin{cases} [0, 0.5] & (i < j) \\ 0.5 & (i \approx j) \\ (0.5, 1] & (i > j) \end{cases} \quad (1)$$

Where $(i < j)$ indicates that preference of ' i ' is less than ' j ', $(i \approx j)$ indicates both items having same preference, and $(i > j)$ indicates ' i ' preferable than ' j '. The value of PR ranges between $[0, 1]$ as evident from (1). Unfortunately there is no explicit PR-based datasets available. Hence we have used the same formula for inducing PRs from rating information as defined in Pujahari and Sisodia (2020b). It is obvious that we will not find PRs for each pair of item for any user. So we have to obtain missing PRs (denoted as: $\hat{\pi}(i, j)_u$). After obtaining missing PRs, we can generate recommendations using (2). Here, $p_{(u,i)}$ is preference score of item i for user u relative to all items:

$$p_{(u,i)} = \frac{\#_{i>j}^u - \#_{i<j}^u}{\#_{i>j}^u + \#_{i\approx j}^u + \#_{i<j}^u} \quad (2)$$

Where, $\#_{i>j}^u$ represents the item count that are less preferred than item i for user u and similar for other terms. The value of this preference score is in between $[-1, +1]$, where $+1$ means the highest preferred item and -1 means lowest preferred item. After obtaining preference scores of all the items for all the users we can generate recommendations.

2.3. Motivation

Good ranking of items is achieved in PR-based RS models, because these models keep items of higher preference at the top of the recommendation list. Although the concept of PR has been studied extensively in personalized recommendation, till date no PR-based RS model is able to capture the change in user preferences with time. From the above section it is also observed that, probabilistic transitive models are good at capturing users' preference change. But, most of these models do not consider user/item features while studying preference shifts. Further ranking quality of these methods cannot be guaranteed, because they only aim at predicting missing ratings. Considering this, we introduced a probabilistic transitive model using Markov random field to study users' preference change with time for generating efficient recommendations.

2.3.1. Research objectives

Considering the motivations stated above the objectives of this research are as follows:

- To build a collaborative filtering model that can keep track of change in users' choices over time.
- To consider user and item features while building model and not solely rely on the ratings.
- To put higher preferred items at the top of the recommendation list because users tend to pick the first few items only.

3. Method

The proposed method is a two fold approach. The first phase contains the point estimation of user preferences using earlier PR-based techniques. The distribution estimation with dynamic user preference is described in the second phase with the help of MRFs. The methodology is similar to Lu et al. (2009). However, we have proposed PR-based approach for improving ranking quality. Table 1 represents the notations/symbols used frequently in this article. The block diagram of the proposed model is shown in Fig. 1. The left part of this figure deals with the point estimation (i.e., predicting preference relations) and the second part deals with the temporal recommendation using distribution estimation.

3.1. PR based factorization model

In typical RS datasets, users provide their preferences in terms of ratings. Let, there be N users and M items in a dataset. Also let, ' $R(N \times M)$ ' be the matrix that contains the ratings given by the users to items. Let, R_i^u denotes the rating given by user ' u ' to item ' i '. Since most of the RS datasets are highly sparse, we have to predict missing ratings in R . Traditional MF technique approximates ' R ' with rank- d approximation. Eq. (3) can be used to obtain predicted rating.

$$\hat{R}_i^u = u_u^T v_i \quad (3)$$

Where, $u(N \times d)$ and $v(M \times d)$ are the user and item latent factor matrices respectively. These matrices can be approximated using the minimization of the following cost function.

$$\min_{u, v} \sum_{(u,i) \in K} (R_i^u - \hat{R}_i^u)^2 + \lambda (\|u\|_F^2 + \|v\|_F^2) \quad (4)$$

Here, λ is the regularization term to alleviate over-fitting, ' K ' contains those user-item pairs, for which rating information is available. A simple solution can be obtained for Eq. (4), using *gradient descent* technique.

As the proposed model is PR-based, we formulate Eq. (4) in terms of PRs. Let, $\pi(i, j)_u \in \Pi$ represent the PR (refer Eq. (1)) of user u towards any pair of item (i, j) . So, now we have to estimate the unknown/missing PR for each item pair (i, j) , i.e. $\hat{\pi}(i, j)_u \in \hat{\Pi}$. So the optimization task can be formulated for PR based RS as:

$$\min_{\hat{\Pi}} \sum_{\hat{\pi}(i,j)_u \in \hat{\Pi}} (\hat{\pi}(i, j)_u - \pi(i, j)_u)^2 \quad (5)$$

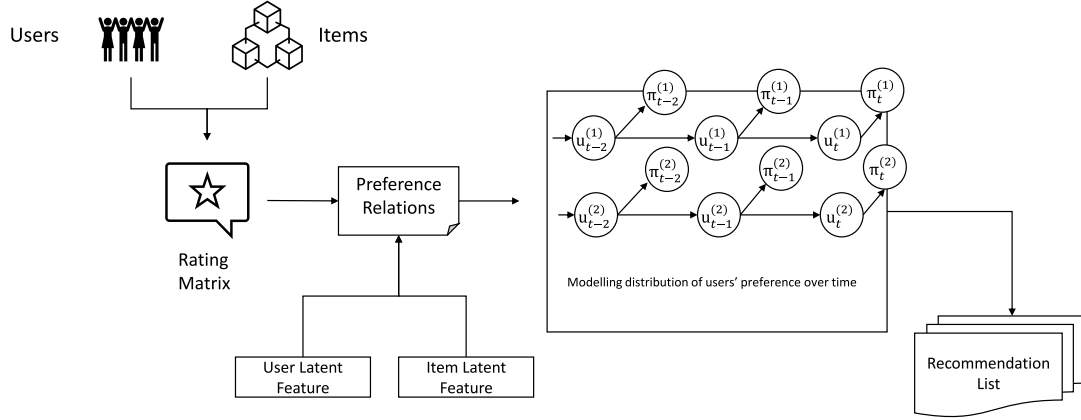
The latent factor matrices(u and v) can be used to predict the missing preference relations in the dataset. Let, u_u be the latent feature vector of u and v_i and v_j are the latent feature vectors of item i and j respectively. The strength of preference of user u for item i and j can be calculated as: $u_u^T v_i$ and $u_u^T v_j$ respectively. Now the PR of user u for item pair (i, j) is $i > j$, if $u_u^T v_i > u_u^T v_j$ and similarly for other relations. The quantitative strength of this PR can be obtained as $u_u^T (v_i - v_j)$. To bound the value of this strength in between 0 to 1, inverse logit function can be used, as given in Eq. (6).

$$\hat{\pi}(i, j)_u = \frac{e^{u_u^T (v_i - v_j)}}{1 + e^{u_u^T (v_i - v_j)}} \quad (6)$$

Table 1

Notations or Symbols used with descriptions.

Notation/Symbol	Description
R	Rating Matrix
R_i^u	Rating of i given by user u
u	Latent feature vector for user u
$\pi(i, j)_u$	Preference relation of u for item pair (i, j)
W_u, W_v	Similarity Matrix of users and items
$f_u(u)$	user latent factor
$U(m)$	neighbors of user m
r_u^t	rating vector of user u at time interval t
$H_u^{(i)}$	Observation operator of respective rows of user latent feature matrix.
$\sum_u^{(i,t)}$	Variance associated with the predicted rating for u at time t

**Fig. 1.** Block diagram of the proposed recommendation model.

The latent features matrices u and v are built by using the following objective function with respect to all known PRs, ' Π '.

$$\min_{u, v} \sum_{\pi(i, j)_u \in \Pi \wedge i < j} (\hat{\pi}(i, j)_u - \pi(i, j)_u)^2 + \lambda (\|u\|_F^2 + \|v\|_F^2) \quad (7)$$

Where, λ is the regularization term and the term $i < j$ ensures no duplicate calculations. This procedure provides a point estimation to the preferences of users, i.e. a quantitative value.

3.2. Distribution estimation using MRF

There may be more information available in addition to user-item latent factor matrices. The information can be user-user similarity, item-item similarity or user/item side information which can be act as a *prior* to an effective user and item latent factor. Fig. 2 represents the distribution estimation for two different scenarios: (a) for item-item similarity, when estimating item factor priors and (b) for user-user similarity when estimating user factor priors. For user latent factors $\{u^{(1)}, \dots, u^{(N)}\}$, let it be denoted as $f_u(u)$. The *Gaussian Markov Random Field* (Besag, 1974) over user latent factor is defined in Eq. (8).

$$f_u(u) \propto \prod_{m, n} e^{-\frac{\alpha}{2} W_u^{(m, n)} \|u^{(m)} - u^{(n)}\|^2} \quad (8)$$

Here, $W_u^{(m, n)}$ denotes the similarity among user m and n and α controls the strength of this prior. If we provide a conditional distribution of user latent factor, it will depend on latent factors of neighborhood users, which is a d -dimensional Gaussian distribution as represented in Eq. (9).

$$u^{(m)} | u^{(-m)} \sim \mathcal{N} \left(\frac{\sum_{n \in U(m)} W_u^{(m, n)} u^{(n)}}{\sum_{n \in U(m)} W_u^{(m, n)}}, \left(\alpha \sum_{n \in U(m)} W_u^{(m, n)} \right)^{-1} \mathbf{I} \right) \quad (9)$$

Where, $U(m)$ represents the neighbors of m and $\mathbf{I}$ is the identity matrix. Eq. (9) model the joint distribution of user-item latent factors through the similarity function W_u . The conditional distribution

represented in (8) infer a joint distribution as given in (9). Here the matrix factorization process discussed above considers the priors are independent of user-item latent factors. Similar to user factor prior as given in (8), we can impose a item-factor prior as:

$$f_v(v) \propto \prod_{i, j} e^{-\frac{\alpha}{2} W_v^{(i, j)} \|v^{(i)} - v^{(j)}\|^2} \quad (10)$$

The joint user-item latent factor prior is yielded by the MRF prior with the spherical Gaussian prior:

$$\text{pri}(u, v) \propto e^{-1/2\lambda(\|u\|_F^2 + \|v\|_F^2)} f_u(u) f_v(v) \quad (11)$$

The '*maximum a posteriori*' (MAP) estimation of u and v in PR-based framework with the above prior leads to the minimization of the following objective function in a PR-framework w.r.t. (7):

$$\min_{u, v} \sum_{\pi(i, j)_u \in \Pi \wedge i < j} (\hat{\pi}_{(i, j)}^u - \pi_{(i, j)}^u)^2 + \lambda (\|u\|_F^2 + \|v\|_F^2) + \alpha (tr(u^T \Delta_u u) + tr(v^T \Delta_v v)) \quad (12)$$

Here, $tr(\cdot)$ represents the trace of the respective matrix, Δ_u denotes the graph Laplacian, which is obtained using the similarity matrix W_u based on the assumption $W_u^{(m, n)} = 0, u = \{1, 2, \dots, N\}$.

$$\Delta_u = D_u - W_u$$

Where, D_u is the diagonal degree matrix and $D_u^{(m, m)} = \sum_n W_u^{(m, n)}$. The difference between similar users m and n are penalized by the term $tr(u^T \Delta_u u)$. The regularization procedure for items can also be defined in the same way.

3.2.1. Integration of side and similarity information

We can integrate the side information (such as user features and item features) in to the regularization model given in (12) with the help of similarity matrix W_u (for users) and W_v (for items). For example:

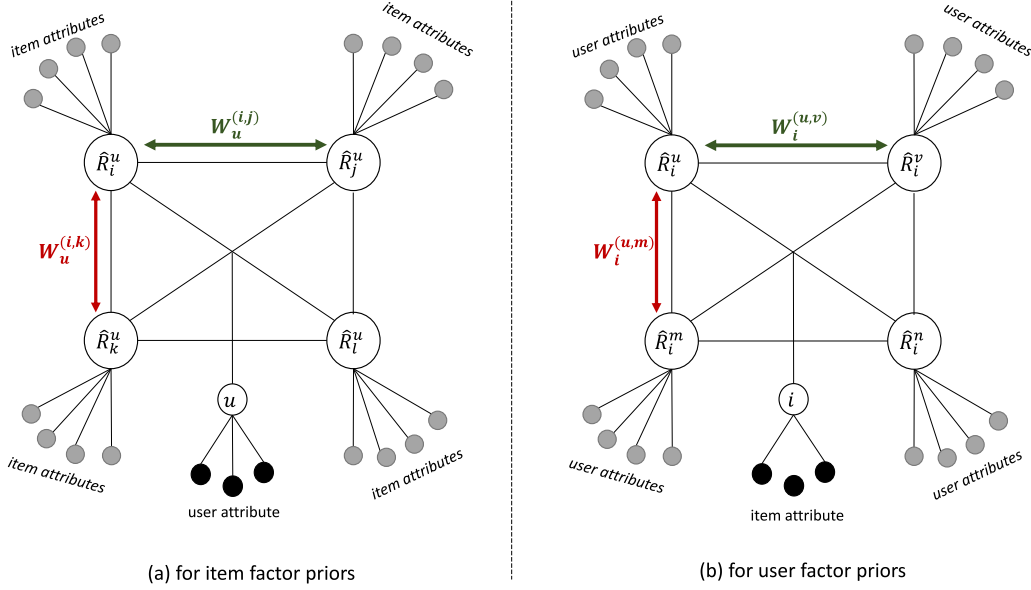


Fig. 2. Distribution estimation for user-user and item-item similarity.

similarity of attributes among two users m and n can be modeled as (if categorical, e.g. gender, occupation):

$$W^{(m,n)} = \delta(c^{(m)}, c^{(n)})$$

Where, $c^{(m)}$ indicates the attribute of user m . For numerical attribute (e.g. age), Gaussian RBF kernel can be used:

$$W^{(m,n)} = e^{-\|c^{(m)}, c^{(n)}\|^2 / \sigma^2}$$

Similar procedure can be done for item attributes. We can also combine these similarity matrices obtained from different sources (say $W_1, \dots, W_m$). One possible approach is by using convex combination: $W = \sum_{i=1}^m \alpha_i W_i$. Example of other sources of information is the neighborhood information, i.e. users having similar taste which can be obtained from their ratings pattern. One of the simplest procedure to obtain this by using *Pearson Correlation*. It can also be applied to item-item correlation. Weak similarity among users/items can be ignored to reduce the computational cost of the optimization process. Both *stochastic gradient descent* and *alternating least square* methods can be used for estimating factors with MF.

3.3. Modeling dynamic user preferences

It is obvious that, item latent factors are known *a priori* that are estimated using traditional MF procedure. But we assumed, user latent factor of each user ' u ' can be considered a random walk with Gaussian noise. Here, (13) is termed as dynamics and (14) is the observation.

$$u_u^{(t)} = u_u^{(t-1)} + w_u^{(t)} \quad (13)$$

$$r_u^{(t)} = H_u^{(t)} u_u^{(t)} + x_u^{(t)} \quad (14)$$

Where, $r_u^{(t)}$ is the rating vector of ' u ' in time interval ' t ', $H_u^{(t)}$ represents the observation operator consists of the respective rows of V (i.e. item latent factor) given in (7); $w_u^{(t)}$ and $x_u^{(t)}$ are the Gaussian noises, where $w_u^{(t)} \sim \mathcal{N}(0, \sum_u)$ and $x_u^{(t)} \sim \mathcal{N}(0, \sigma_0^2 \mathbf{I})$.

We have followed the same inference procedure as discussed by Lu et al. (2009) (i.e. by using *Kalman Filtering* (KF)) for the dynamic model discussed above to estimate user latent factors. KF basically takes sequential rating states $\{\dots, r_u^{(t-2)}, r_u^{(t-1)}, r_u^{(t)}\}$ as observation and estimates the optimal state at time t , represented by $\hat{u}_u^{(t)}$ with the

associated variance $\sum_u^{(t)}$. During each step of KF at time ' t ', estimation of $(\hat{u}_u^{(t-1|t-1)}, \sum_u^{(t-1|t-1)})$ from earlier step is updated through new observation $r_u^{(t)}$ as:

$$\hat{u}_u^{(t|t-1)} = \hat{u}_u^{(t-1|t-1)}, \sum_u^{(t|t-1)} = \sum_u^{(t-1|t-1)} + \sum_u \quad (15)$$

$$\sum_u^{(t|t)} = \sum_u^{(t|t-1)} - \sum_u^{(t|t-1)} H_T^{(t)} (H \sum_u^{(t|t-1)} H_T^{(t)} + \sigma_0^2 \mathbf{I})^{-1} H^{(t)} \sum_u^{(t|t-1)} \quad (16)$$

$$\hat{u}_u^{t|t} = \hat{u}_u^{t|t-1} + \sum_u^{t|t} H_T^t \sigma_0^{-2} (r_u^t - H^t \hat{u}_u^{t|t-1}) \quad (17)$$

Here, Eq. (15) represents 'time-update' and Eq. (16) & Eq. (17) represents measurement update of user latent factor. We can also model a joint filtering technique for both user and item latent factors. But in this study we limited our study to dynamic user preferences only. We can integrate the latent factor model discussed in Section 3.2 in this dynamic observation to generate recommendations.

3.4. Unifying latent factor and dynamic user preference

Let us consider the new prior for user latent factor that is capable of representing point estimation as well as dynamic preferences, $u^t \equiv \{u_1^t, \dots, u_N^t\}$. Also let, $\theta = \{\lambda, \alpha, \sum_u, \sigma_0\}$ unifies all parameters discussed in earlier sections. For each time step t , prior of p^t is derived from two separate sources: i.e. dynamic user preferences (discussed in Section 3.3) and latent factor with similarity model (discussed in Section 3.2).

The dynamic preference of user can be modeled using probability $p(u^t | u^{t-1}; \theta)$. This will penalize the large deviation among p^t and the prediction at $t-1$. For the user random walk (as discussed in earlier subsection) we can have:

$$p(u^t | u^{t-1}; \theta) = \prod_{u=1}^N p(u_u^t | u_u^{t-1}; \theta) \\ \propto \prod_{u=1}^N e^{-1/2(u_u^t - u_u^{t-1})^T \sum_p^{-1} (u_u^t - u_u^{t-1})} \quad (18)$$

Further, the similarity based latent factor model can also be expressed by time-varying Gaussian MRF prior $p(u^t; W^{p,t}, \theta)$, where $W^{p,t}$ represents the similarity among users at a given time ' t '.

$$p(u^t; W^{p,t}, \theta) \propto e^{-1/2\alpha \sum_{m,n} W_{u,u}^{mn} \|u_m^t - u_n^t\|^2} \quad (19)$$

Here, $W_{u,t}^{mn}$ represents the (m, n) entry of similarity weight matrix $W_{u,t}$. Now unification of both dynamic and similarity based priors can be obtained by the product:

$$pri(u^t) \propto p(u^t | u^{t-1}; \theta) p(u^t; W_{u,t}, \theta) \quad (20)$$

If we have given the user latent factor, the likelihood of the ratings (predicted) is:

$$p(r^t | u^t; \theta) = \prod_{u=1}^N u(r_u^t | u_u^t; \theta) \prod_{u=1}^N e^{-\frac{\|r_u^t - H_u^t u_u^t\|^2}{2\sigma_0^2}} \quad (21)$$

The complete likelihood of u^t from $\tau = 1$ to t and r^t from $\tau = 1$ to t , can be obtained as (provided $r^t \equiv \{r_1^t, \dots, r_N^t\}$):

$$p(\{u^t\}_{\tau=1}^t, \{r^t\}_{\tau=1}^t, \{W_{u,t}\}_{\tau=1}^t; \theta) = \prod_{\tau=1}^t p(u^t | u^{t-1}; \theta) p(r^t | u^t; \theta) p(u^t; W_{u,t}, \theta) \quad (22)$$

3.5. Inference procedure

We cannot estimate u^t for each user u separately due to the integration of correlation/side information $W_{u,t}$. Due to the concatenated state vector, the implementation of KF update:

$$\bar{u}^t = [(u_1^t)^T, \dots, (u_N^t)^T]^T \quad (23)$$

and the respective co-variance matrix $\bar{\Sigma}^t (\in \mathcal{R}^{d \times d})$. The probability $p(\bar{u}^t | \{r^t\}_{\tau=1}^t; \{W_{u,t}\}_{\tau=1}^t, \theta)$ can be estimated by recursively (similar to Eqs. (15) and (16)) following the two steps:

$$\hat{u}^{t|t-1} = \hat{u}^{t-1|t-1}, \bar{\Sigma}^{t|t-1} = \bar{\Sigma}^{t-1|t-1} + \Sigma_u \quad (24)$$

$$p(\bar{u}^t | \{r^t\}_{\tau=1}^t; \{W_{u,t}\}_{\tau=1}^t, \theta) \propto p(\bar{u}^t | \text{est}^{t-1}, \theta) p(\bar{u}^t; W_t, \theta) p(r^t | \bar{u}^t; \theta) \quad (25)$$

Here, $\text{est}^{t-1} = \{\hat{u}^{t-1|t-1}, \bar{\Sigma}^{t-1|t-1}\}$ are the obtained mean and covariance of $\bar{u}^t$ at time $t-1$. Also, Eq. (24) is the ‘time-update’ and Eq. (25) is the measurement update similar to as defined in Section 3.3. The inverse computation of such a large covariance matrix is computationally expensive. To avoid this, we have used ‘mean field approximation’ MFA (Jaakkola, 2001) to infer at each time interval. Using MFA we have the following factorized probability v^t to approximate $p(\bar{u}^t | \{r^t\}_{\tau=1}^t; \{W_{u,t}\}_{\tau=1}^t, \theta)$.

$$v^t(u^t) \equiv \prod_{u=1}^N v_u^t(u_u^t) \quad (26)$$

Considering this assumption, the co-variance of the posterior distribution can be seen as a combination of individual ones:

$$\bar{\Sigma}^{t|t} = \{\sum_1^{t|t} \dots \sum_N^{t|t}\} \quad (27)$$

Using minimum KL-divergence we can find the probability $v^t(p^t)$:

$$v_{*}^t(u^t) = \underset{v^t}{\text{argmin}} KL(v^t(u^t) \| p(u^t | \{r^t\}_1^t; \{W_{p,t}\}_1^t, \theta)) \quad (28)$$

This can be remodeled as:

$$\max_{v^t} H(v^t) + \mathcal{E}_{v^t} [\log p(u^t | \{r^t\}_1^t; \{W_{p,t}\}_1^t, \theta)] \quad (29)$$

Where, $\mathcal{E}_{v^t}$ represents the expectation w.r.t. v^t . We can find a local optimum by running 30 and 31 iteratively for each user u :

$$v_u^t(u_u^t) \leftarrow \frac{1}{\Psi_u} e^{\mathcal{E}_{v^t} [\log p(u^t | \{r^t\}_1^t; \{W_{p,t}\}_1^t, \theta) | u_u^t]} \quad (30)$$

Here, $\mathcal{E}_{v^t}[\cdot]$ is the expectation of the respective term w.r.t. v^t conditioned on u_u^t and Ψ_u is the normalization constant for v_u^t . It can be noted that the posterior of $p(u^t | \{r^t\}_1^t; \{W_{p,t}\}_1^t, \theta)$ is Gaussian.

$$\Psi_u = \int_{u_u^t} e^{\mathcal{E}_{v^t} [\log p(u^t | \{r^t\}_1^t; \{W_{p,t}\}_1^t, \theta) | u_u^t]} du_u^t \quad (31)$$

3.6. Recommendation procedure

Here we will provide a step by step approach to generate recommendation using the proposed model with the help of an algorithm. Before mentioning the algorithm we will discuss two typical scenarios of any RSs. The first one is the case of ‘new users’. In case of new users the initial user latent factor can be obtained using the covariates. The user latent factor is derived from the conditional distribution of the MRF prior, as discussed in (12). We can do this through regularized version of the nearest neighbor interpolation.

$$\bar{u}_m = \frac{\sum_{n \in \mathcal{U}^*(m)} W_u^{(m,n)} u_{(n)}}{\sum_{n \in \mathcal{U}^*(m)} W_u^{(m',n)} + \lambda/\alpha} \quad (32)$$

The second case is to handle new items. It is obvious that, the problem arises when we have too few ratings and also when the item latent factor changes with time. The updation of item latent factor at each time interval t can be found out by finding the approximation to the following:

$$v_i^t = \underset{v}{\text{argmin}} \left\{ \sum_{\tau=1}^t \sum_{F^{\tau}(i*)} \frac{\|r_{(u*)}^{\tau} - u_u^{\tau T} v\|^2}{2\sigma^2} + \sum_{\tau=1}^t \sum_{j \in \mathcal{U}^*(i*)} \frac{\alpha_i W_v^{(i,j)} \|v - v_j^{\tau}\|^2}{2} \right\} \quad (33)$$

Where, $F^{\tau}(i*)$ represents the users who have commonly rated item i at time τ , $\mathcal{U}^*(i*)$ represents the items nearest neighbors of i w.r.t. W_v . Considering these we can generate recommendation of new items for any user u with capturing changing preferences over time. Algorithm 1 shows the step by step procedure for generating Top-K recommendations for any user.

Algorithm 1 Proposed PrefDMRF Algorithm

- 1: **Input:** $\mathbf{R}$, the rating matrix with time sequence, i.e. $\{R_1, \dots, R_T\}$ and ‘ R_0 ’ rating before time.
- 2: **Output:** Top-K recommendations
- 3: Infer PRs from the given rating matrix.
- 4: Initialize user latent factors $\{\hat{u}_1^{0|0}, \dots, \hat{u}_N^{0|0}\}$ and obtain item latent factors $\{v_1, \dots, v_M\}$ using (7) over ‘ R_0 ’
- 5: $t \leftarrow 1$; For all user ‘ u ’ set initial variances $\{U_u^{0|0}\}$
- 6: **repeat** ▷ For time sequence $t=1$ to T
- 7: **for** all $u \in U$ **do** ▷ Kalman Filtering update
- 8: Update $[\hat{u}_u^{t|t}, \Sigma_u^{t|t}]$ using Eq. (15), Eq. (16) and Eq. (17)
- 9: **end for**
- 10: **for** $u \leftarrow 1$ to N **do** ▷ Mean Field Approximation
- 11: $v_u \leftarrow \mathcal{N}(\hat{u}_u^{t|t}, \Sigma_u^{t|t})$
- 12: **end for**
- 13: **repeat**
- 14: $v_u(\bar{u}_u^t) \leftarrow \frac{1}{\Psi_u} e^{\mathcal{E}_{v^t} [\log p(\{\bar{u}_u^t\}_1^N | r_u^t, W_{u,t}) | \bar{u}_u^t]}$
- 15: **until** converged
- 16: **return** mean and covariance of each v_u
- 17: Update the item latent factors
- 18: $t = t + 1$
- 19: **until** $t=T$
- 20: **for** each $u \in U$ **do** ▷ Generating recommendations
- 21: **for** each unknown item pair (i, j) **do**
- 22: Calculate $\hat{\phi}_{(i,j)}^u$ using (6)
- 23: **end for**
- 24: Calculate score of any item i for user u using (2)
- 25: **end for**
- 26: List out Top-K items according to obtained score for any user u

Here the user-item preferences (i.e. ratings) are given in time sequence manner. We will provide some initial ratings (i.e. before time 0) and sequence of ratings up to time T . The next step is the initialization of user latent factor for each time sequence. Since we

assumed item latent factor matrix to be static through out the entire optimization process, it is initialized with random real values with Gaussian distribution. The inference of the user and item latent factors with respect to time is done using Kalman filtering as written in Step 7 to 9 of Algorithm 1. Next step we have used the MFA to infer user latent for each time step by updating the mean and co-variance. After obtaining approximation, we can find out the predicted rating for any unknown item for each user. Also, since our proposed model uses PR instead of ratings, the predicted ratings are converted to PRs, as represented in step 20 to 23. Using the predicted PRs, the preference score of items are generated for any user and based on the score the recommendation is generated. The preference score helps in placing higher preferred items at the top of the recommended list.

3.6.1. Computational complexity

The major computationally expensive part of Algorithm 1 is the learning the user latent feature vector 'u'. During that process, the Kalman Filtering update will require much computational task. The Kalman Filtering step requires computation of inverse of covariance matrix mentioned in Eq. (25). But using mean field approximation we can estimate the update in a reasonable time. The step 11 of Algorithm 1 requires multiplication of three equal size matrices (equivalent to user latent features). Hence this update will cost $O(n^3)$. The second part is the update via inverse part, which is represented in Step 14. Considering the number of users to be n and number of items to be m this step requires a series of matrix multiplication. Step 11 of this algorithm will cost $O(n^2 m)$, where as Step 14 will cost $O(n^3)$. Considering all these scenarios, the final computational complexity of the recommendation procedure will be $O(n^3 + n^2 m + n^3)$ where n is the number of users.

4. Experimental evaluation

To evaluate the effectiveness of our proposed method, i.e. PrefDMRF, we have used three real world datasets belong to the domain of movies, songs. We will discuss the datasets in brief in the next part which is followed by the results.

4.1. Datasets

We have used the following datasets for the experimental verification of the proposed model.

1. *NetFlix*: This dataset (Koren, 2009) was a part of NetFlix competition.¹ It contains approximately 100 million ratings given by 480 000 users to 17 770 movies.
2. *MovieLens-1M*: This dataset is released by *grouplens* (Harper & Konstan, 2015) and is one of the widely used datasets for RS evaluation. It contains around 6040 users given ratings to 3706 number of movies. The ratings is about 1 million.
3. *Last.fm*: The dataset was extracted from the internet music website Last.fm (Bertin-Mahieux et al., 2011). It contains 992 users who have listened/used 177 000 artists' albums/songs between 2005 to 2009.

Since there are some infrequent users in all the above three datasets, we have removed some of them. The process is similar to as defined by Zhang et al. (2018).

Since the proposed model, i.e. PrefDCRF uses user and/or item side information during model building, we have extracted the information available in each dataset. The ML-1M dataset contains user information like age, gender and occupations and movie information like 18 different genres. Some of these are categorical values, and hence they are

Table 2

Performance of the proposed model by varying Latent features.

Dataset	Latent Features (50)		Latent features (80)		Latent features (100)	
	RMSE	MAE	RMSE	MAE	RMSE	MAE
ML-1M	0.7641	0.7085	0.7334	0.7126	0.7316	0.7077
NetFlix	0.7925	0.7328	0.7640	0.7328	0.7589	0.7306
Last.fm	0.8280	0.8075	0.8036	0.7855	0.8013	0.7789

converted to vector format using embedding technique. For simplicity, for each user and movie, we have used a ' D ' dimensional binary vector. For example: age is converted to five categories (<18, 18–25, 25–30, 30–35 and >35) and user belong to any of these category is represented as 1 and leaving others as zeros. Similar procedure has been done for gender and occupation attributes. The *NetFlix* dataset do not contain side information about users or movies. But, we have gathered features of the movies from the IMDB database manually and incorporated into the model using the similar procedure discussed above. Last.fm dataset contains information of users like, age gender and country. These were used as side information the proposed model.

To evaluate the changing preferences of users over time, we have used different procedures for each of the datasets discussed above. The rating file of ML-1M dataset has four parts: user_id, movie_id, rating,time_stamp over 3 years. The ratings were given in a scale of 1 to 5. Based on the users' viewing history, we split the dataset into 6 time periods (followed the similar procedure used by Rafailidis & Nanopoulos, 2016). Each time slot corresponds to users' behavior of 6 months. Similarly, Last.fm and NetFlix datasets were splitted into 9 time periods (over 54 months) and 7(over 42 months) time periods respectively.

4.2. Parameter selection

The major hyper parameters that regulates the performance of the proposed model are: the dimension of the latent feature vector (D), Number of neighbors chosen for calculating weight matrix(W) and α that controls the strength of the similarity information prior. The performance of the proposed model by varying the dimension of latent factor is given in Table 2 using RMSE and MAE measures. We can see that the RMSE and MAE decreases(lower is better) with increase in dimension but reaches saturation after 80. Hence we have taken the dimension to be 80 for the user latent feature of the proposed method. Next, the number of neighbors chosen for calculating weight matrix is '5' in our case. Taking a value larger than five will not improve performance significantly for the proposed method. The last but the important one is the value of α , which is usually a small value taken for regulating the similarity information. We have taken the value of 0.05 for this. Apart from these the regularization parameter used for point estimation (λ) is also taken as 0.05.

Before comparing the proposed models with related state-of-the-art model, we first evaluated the proposed model using RMSE and MAE measure discussed in the previous section. Since, we are using preference relation instead of ratings, here the errors are reported by comparing the PR scores. Table 2 contains the RMSE and MAE values of the proposed model using all the three datasets for different latent factors. Here the latent factor (numbers) is the dimension of the user latent factor matrices during the model building. The key observation from this table is that the RMSE and MAE value decreases (lower is better) when we increase the dimension, which is obvious. Further, we are getting less performance for the Last.fm dataset because of the high sparseness and low feature information available.

¹ www.netflixprize.org

4.3. Metrics for comparison

The proposed RS model ‘PrefDMRF’ evaluated as well as compared in terms of accuracy as well as ranking. Measures like, Precision(P), Recall(R) and F1-score(F1) of the recommendation is used for evaluating accuracy of the recommendation. The value of P, R and F1 can be obtained for a recommendation list can be obtained by using Eq. (34), Eq. (35) and Eq. (36) respectively.

$$P = \frac{|R \cap U|}{|R|} \quad (34)$$

$$R = \frac{|R \cap U|}{|U|} \quad (35)$$

$$F1 = \frac{2 \times P \times R}{(P + R)} \quad (36)$$

Here ‘R’ is the set of recommended items and ‘U’ is the set of actual used/consumed items in the test dataset. These measures test the recommendation accuracy of any RS model. These metrics do not consider the ordering of items in the recommendation list. It may happen that, the items present at lower position in the recommended list are highly preferable to a user. Hence ordering of items in the recommendation list is important. To evaluate this, we used two ranking based measures NDCG@K and Precision@K, where ‘K’ represents the position.

The idea behind the graded relevance is that the higher preferred items appearing lower in the recommended list should be penalized. In *Discounted Cumulative Gain* (DCG) (Järvelin & Kekäläinen, 2002) measure at a particular position ‘K’ (refer Eq. (37)) the graded relevance value is reduced logarithmically proportional to the recommended list. Here ‘K’ is the number of recommended items. Hence the ‘DCG’ value will be different, when we change the ‘K’ value. So a ‘Normalized DCG’@K (refer Eq. (38)) is often preferable in case of recommender system evaluation.

$$DCG@K = \sum_{i=1}^K \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (37)$$

$$NDCG@K = \frac{DCG@K}{IDCG@K} \quad (38)$$

Where, $IDCG@K$ is the maximum possible DCG can be achieved at a particular position ‘K’, means all the items are correctly placed in the recommended list according to their preferences.

4.4. Results

Experiments were performed using time intervals for each of the dataset (discussed in Section 4.1) for evaluating the dynamic user preferences. For comparison we have used seven baselines: TADCF (Wang et al., 2022), RSTRC (Ahmadian et al., 2022), TPRC (Zhao et al., 2022), NHMM (Carroll et al., 2022), DTRS (Zhang et al., 2021), FuseRec (Narang et al., 2021), and MF-LOD (Natarajan et al., 2020). The parameters of the respective algorithms are chooses so that to obtain best results. For unbiased comparison each algorithm is tested using same set of training and test data and also using same time intervals. The dynamic user preference is evaluated by predicting the preferable items at time $t + 1$ when the preferences of users at time t is known.

Tables 3–5 contains the performances of the specified algorithms using Netflix, MovieLens, and Last.fm datasets respectively. All the methods used the ratings available at time slot t to predict the preference of users at $t + 1$. The process will continue until the prediction for all the time slots are recorded. As mentioned in the tables, the values are obtained when number of items recommended is 5 and 10. For training we have taken 70% of the ratings and the remaining for testing. For calculating P, R and F-1 score we have obtained the truth values from the actual ratings given by user. Means, items having higher ratings were added as a truth values for measuring performance with the predicted ones. We can see from these tables that, the proposed

Table 3

Comparison of performance for Netflix.

Models	Recommending 5 items			Recommending 10 items		
	Precision	Recall	F1-Score	Precision	Recall	F1-Score
FuseRec	0.2156	0.2238	0.2434	0.2118	0.2265	0.2623
DTRS	0.2567	0.2646	0.2877	0.2464	0.2598	0.2945
RSTRC	0.2428	0.2496	0.2713	0.2375	0.2587	0.2856
TPRec	0.2849	0.2967	0.3136	0.2788	0.2953	0.3201
TADCF	0.2927	0.3141	0.3354	0.2913	0.3036	0.3164
MF-LOD	0.2865	0.3011	0.3132	0.2689	0.2810	0.2944
NHMM	0.3277	0.3369	0.3512	0.3124	0.3268	0.3417
PrefDMRF	0.3383	0.3510	0.3746	0.3249	0.3390	0.3698

Table 4

Comparison of performance for MovieLens.

Models	Recommending 5 items			Recommending 10 items		
	Precision	Recall	F1-Score	Precision	Recall	F1-Score
FuseRec	0.2369	0.2538	0.2765	0.2219	0.2478	0.2637
DTRS	0.2567	0.2646	0.2877	0.2464	0.2598	0.2945
RSTRC	0.2511	0.2732	0.2968	0.2473	0.2605	0.2933
TPRec	0.3036	0.3142	0.3387	0.2951	0.3093	0.3179
TADCF	0.3007	0.3123	0.3364	0.2945	0.3058	0.3156
MF-LOD	0.2876	0.3011	0.3132	0.2765	0.2877	0.2938
NHMM	0.3422	0.3570	0.3659	0.3217	0.3328	0.3435
PrefDMRF	0.3586	0.3657	0.3892	0.3463	0.3517	0.3625

Table 5

Comparison of performance for Last.fm.

Models	Recommending 5 items			Recommending 10 items		
	Precision	Recall	F1-Score	Precision	Recall	F1-Score
FuseRec	0.1916	0.2133	0.2280	0.1857	0.2038	0.2179
DTRS	0.2126	0.2288	0.2469	0.2003	0.2124	0.2256
RSTRC	0.2165	0.2279	0.2434	0.2103	0.2258	0.2439
TPRec	0.2293	0.2467	0.2614	0.2231	0.2355	0.2472
TADCF	0.2318	0.2471	0.2603	0.2299	0.2410	0.2573
MF-LOD	0.2178	0.2240	0.2335	0.2103	0.2215	0.2291
NHMM	0.2575	0.2644	0.2811	0.2423	0.2598	0.2719
PrefDMRF	0.2699	0.2803	0.2954	0.2611	0.2740	0.2908

model performing better than others in all the three datasets, although the performance of NHMM (Carroll et al., 2022) is relatively closer to the performance of the proposed model. Further, the performance of all the methods are relatively low for Last.fm dataset due to the larger time slots and lesser number of training data for each time slots. Similarly, due to the lesser number of time slots and proper rating information the performance of all the methods is better in comparison with the other three datasets.

To show the performance of each method with respect to each test month we conducted separate experiments for each time slots. Fig. 3 shows the performance of respective methods over different test months for all the datasets. Since each dataset is splitted into different no of time slots the months in the x-axis is different for all the datasets. Here also we can observe that the proposed method, i.e. PrefDMRF is performing better than others for each test months. This is because, the proposed method uses neighborhood information along with side information while optimizing the user-item latent factors. The performance of NHMM model is very close to the PrefDMRF model for MovieLens dataset (refer Fig. 3(b)). This may be due to the lesser number of time slots and lesser user preference drifts. Due to the uncertainty in the users’ preference changes, the average precision increases/decreases during test months. Among other models TPRC is performing a constantly, because this method also uses information similar to the proposed model.

The performance of all the methods using ranking measures is given in Table 6 along with standard deviation over ten runs. The results

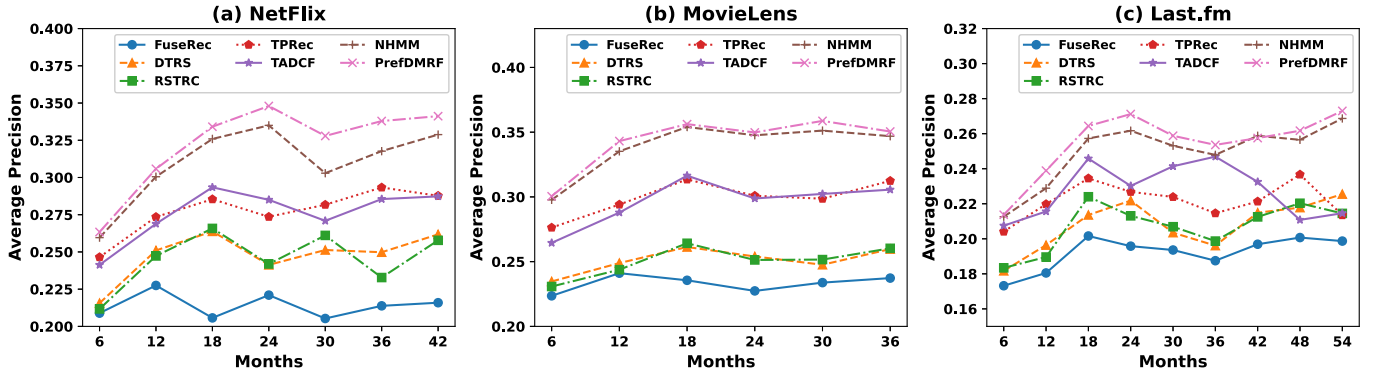


Fig. 3. Average precision of respective methods for the test months.

Table 6

Performance comparison of different methods for ranking quality.

Methods	NetFlix		MovieLens		Last.fm	
	NDCG	Precision	NDCG	Precision	NDCG	Precision
FuseRec	0.3076 $\pm$ 0.0013	0.2853 $\pm$ 0.0008	0.3416 $\pm$ 0.0013	0.3329 $\pm$ 0.0016	0.2796 $\pm$ 0.0007	0.2643 $\pm$ 0.0009
DTRS	0.3349 $\pm$ 0.0015	0.3268 $\pm$ 0.0012	0.3568 $\pm$ 0.0013	0.3427 $\pm$ 0.0016	0.3055 $\pm$ 0.0005	0.2977 $\pm$ 0.0014
RSTRC	0.3462 $\pm$ 0.0006	0.3399 $\pm$ 0.0009	0.3712 $\pm$ 0.0011	0.3587 $\pm$ 0.0013	0.3209 $\pm$ 0.0016	0.3165 $\pm$ 0.0008
TPRec	0.3738 $\pm$ 0.0005	0.3556 $\pm$ 0.0018	0.3985 $\pm$ 0.0011	0.3812 $\pm$ 0.0015	0.3346 $\pm$ 0.0013	0.3281 $\pm$ 0.0014
TADCF	0.3877 $\pm$ 0.0009	0.3643 $\pm$ 0.0011	0.4047 $\pm$ 0.0012	0.3978 $\pm$ 0.0006	0.3560 $\pm$ 0.0014	0.3451 $\pm$ 0.0017
NHMM	0.4018 $\pm$ 0.0017	0.3982 $\pm$ 0.0021	0.4329 $\pm$ 0.0014	0.4272 $\pm$ 0.0012	0.3955 $\pm$ 0.0017	0.3876 $\pm$ 0.0011
PrefDMRF	0.4278 $\pm$ 0.0007	0.4126 $\pm$ 0.0013	0.4587 $\pm$ 0.0016	0.4302 $\pm$ 0.0009	0.4136 $\pm$ 0.0007	0.4011 $\pm$ 0.0016

Table 7

Performance of the proposed model with and without side information.

Measures	PrefDMRF with side information			PrefDMRF without side information		
	NetFlix	ML-1M	Last.fm	NetFlix	MovieLens	Last.fm
Precision	0.4179	0.4325	0.4067	0.382	0.4011	0.3926
Recall	0.4301	0.4583	0.4423	0.4039	0.4260	0.4126
F1	0.4239	0.4450	0.4237	0.3926	0.4131	0.4023

given in table were obtained using the same set of training and test data with top-10 recommendations (i.e., $K=10$). We can observe from this table that, the proposed PrefDMRF model clearly outperformed all other methods by a significant margin including NHMM model. This is because PR-based models are good at ordering of items in a recommended list.

4.5. Ablation study

Here we have studied the contributions of different components of the proposed model. For that we have considered different variants of the proposed model. The first thing that we considered in the proposed model is the use of PR instead of ratings, which improves the ranking quality. Hence, we compared the results obtained using both PRs and ranking based on top-10 recommendation tasks for all the datasets. For temporal recommendations we have followed the same time intervals (6 months) as discussed in Section 4.4. Fig. 4 shows the performance of the proposed model with respect to NDCG and Precision measure. The values given for each dataset is the average value obtained for all the time intervals of the respective dataset. While considering ratings, the recommended items are ordered in the non-increasing order of the predicted ratings. We can observe from this figure that PR-based models are better at ranking, which is also validated in Liu et al. (2017) and Pujahari and Sisodia (2020b).

The second variant that we considered is the proposed model with fusion of ratings with side information and without it. For this we have used the accuracy base measures using all the three datasets. The default model will integrate the side information with the MRF during distribution estimation of users' preference shifts. The proposed model

without considering side information will ignore the side information and only consider the distribution of users' preferences over different time periods. Table 7 shows the performance of the proposed model with and without side information. It can also be observed from this table that the performance of the proposed model with side information is better for all the measure and all the datasets.

5. Discussion

One of the significant observation from the previous section is that the probabilistic transitive models are better at capturing user preference drifts. It has also been noted that the integration of weight matrix 'W' plays a significance role towards the quality of recommendations. Because, through 'W' we can provide as many information we want, i.e. correlation among users and/or items, user and item feature information etc. One of the main advantage of the proposed model is that, the user and item latent factors are optimized in line with the weight matrix, which is effective in case of cold-start problems. This is because we can keep track of users' preference shifts to various attributes/features of items with the passage of time. Regarding the ranking quality, PR-based models are better than traditional rating based models. Among PR-based models, PrefCRF (Liu et al., 2017) was a promising approach, which also uses MRF to estimate the distribution of user preferences. But it does not model change in users preferences with time. Also, this model uses point estimation (using PR-based MF procedure) and distribution estimation (using MRF) separately for generating recommendations. Whereas, in the proposed model the user and item latent factors are optimized simultaneously along with other hyper parameters, and hence is more efficient.

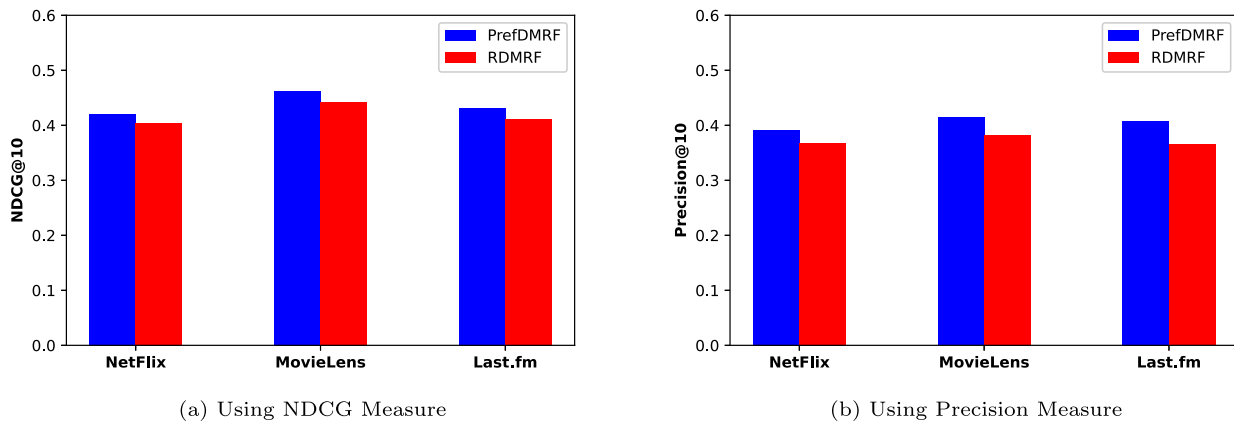


Fig. 4. Performance comparison of PR-based and rating-based versions of proposed model.

Another major problem in studying users' behaviors over time is the unavailability of the preferable datasets. This is one of the limitations of the proposed model. Most of the RS models including the proposed model (PrefDMRF), that are capable of handling the dynamic preferences of users, use the datasets that are not standardized for studying change in users behaviors over time. Also, these dataset are not collected for a longer period of time, i.e. the maximum duration dataset discussed in this paper is 'Last.fm' having ratings given over a period of 54 months. Hence, to study users' change in preferences, we need ratings or preferences for much longer period. Also, side information is not present in every datasets. Hence, a dataset having user-item interaction information over a long period, is required for more efficient evaluation of a RS model, considering dynamic user preferences. This is another limitation of our proposed model.

6. Conclusion

As there are plenty of information available, users tend to change their tastes. This makes the job of recommender systems much more difficult. Most of the RSs that capture dynamic user preferences, uses time slots as a variable in a latent factor based model, that can represents users preference drifts. However, when there are few ratings available, it is hard to predict the next set of preferable items for a user. In that case, user and item features can be effective, which is verified in the experimental results. Further, the user and item latent factor matrices are co optimized along with other hyper parameters that includes the similarity information and side information, during the model building. This makes the system more robust to predict user preference change with time. Use of PRs instead of ratings improves the ranking quality of the system. In future work, we would like to explore deep learning methods to build the latent factor model to study the users' behavior over time. There are Spatio-temporal models are available using deep learning, but not in the domain of RS. Hence, there can be a possibility of enhancement in existing time-dependent RS models.

CRedit authorship contribution statement

Abinash Pujahari: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Writing – original draft. **Dilip Singh Sisodia:** Resources, Supervision, Validation, Visualization, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Ahmadian, S., Joorabloo, N., Jalili, M., & Ahmadian, M. (2022). Alleviating data sparsity problem in time-aware recommender systems using a reliable rating profile enrichment approach. *Expert Systems with Applications*, 187, Article 115849. <http://dx.doi.org/10.1016/j.eswa.2021.115849>.
- Ait Hammou, B., Ait Lahcen, A., & Mouline, S. (2019). An effective distributed predictive model with matrix factorization and random forest for big data recommendation systems. *Expert Systems with Applications*, 137, 253–265. <http://dx.doi.org/10.1016/j.eswa.2019.06.046>.
- Bertin-Mahieux, T., Ellis, D. P., Whitman, B., & Lamere, P. (2011). The million song dataset. In *Proceedings of the 12th international conference on music information retrieval*.
- Besag, J. (1974). Spatial interaction and the statistical analysis of lattice systems. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36, 192–225.
- Brun, A., Hamad, A., Buffet, O., & Boyer, A. (2010). Towards preference relations in recommender systems. In *Workshop on preference learning, European conference on machine learning and principle and practice of knowledge discovery in databases*, vol. 51.
- Carroll, M. D., Dragan, A., Russell, S., & Hadfield-Menell, D. (2022). Estimating and penalizing induced preference shifts in recommender systems. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, & S. Sabato (Eds.), *PMLR volume 162 of proceedings of machine learning research*, (pp. 2686–2708).
- Desarkar, M. S., Saxena, R., & Sarkar, S. (2012). Preference relation based matrix factorization for recommender systems. In *International conference on user modeling, adaptation, and personalization* (pp. 63–75). Springer.
- Gharahighehi, A., & Vens, C. (2023). Diversification in session-based news recommender systems. *Personal and Ubiquitous Computing*, 27, 5–15.
- Harper, F. M., & Konstan, J. A. (2015). The movielens datasets: History and context. *ACM Transactions on Intelligent Systems and Technology*, 5, 19:1–19:19. <http://dx.doi.org/10.1145/2827872>.
- Harshvardhan, G., Gourisaria, M. K., Rautaray, S. S., & Pandey, M. (2022). Ubmtr: Un-supervised boltzmann machine-based time-aware recommendation system. *Journal of King Saud University-Computer and Information Sciences*, 34, 6400–6413.
- Hazrati, N., & Ricci, F. (2022). Recommender systems effect on the evolution of users' choices distribution. *Information Processing & Management*, 59, Article 102766.
- Jaakkola, T. (2001). Tutorial on variational approximation methods. In *Advanced mean field methods: theory and practice*, vol. 129.
- Jalali, S., & Hosseini, M. (2022). Collaborative filtering in dynamic networks based on deep auto-encoder. *The Journal of Supercomputing*, 78, 7410–7427.
- Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20, 422–446. <http://dx.doi.org/10.1145/582415.582418>.
- Koren, Y. (2009). Collaborative filtering with temporal dynamics. In *Proceedings of the 15th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 447–456). New York, NY, USA: ACM, <http://dx.doi.org/10.1145/1557019.1557072>.
- Koren, Y., Rendle, S., & Bell, R. (2021). Advances in collaborative filtering. In *Recommender systems handbook* (pp. 91–142).
- Lin, A., Wang, J., Zhu, Z., & Caverlee, J. (2022). Quantifying and mitigating popularity bias in conversational recommender systems. In *Proceedings of the 31st ACM international conference on information & knowledge management* (pp. 1238–1247).

- Liu, S., Li, G., Tran, T., & Jiang, Y. (2017). Preference relation-based Markov random fields for recommender systems. *Machine Learning*, 106, 523–546. <http://dx.doi.org/10.1007/s10994-016-5603-7>.
- Lo, Y., Liao, W., Chang, C., & Lee, Y. (2018). Temporal matrix factorization for tracking concept drift in individual user preferences. *IEEE Transactions on Computational Social Systems*, 5, 156–168. <http://dx.doi.org/10.1109/TCSS.2017.2772295>.
- Lu, Z., Agarwal, D., & Dhillon, I. S. (2009). A spatio-temporal approach to collaborative filtering. In *Proceedings of the third ACM conference on recommender systems* (pp. 13–20). New York, NY, USA: ACM, <http://dx.doi.org/10.1145/1639714.1639719>.
- Milano, S., Taddeo, M., & Floridi, L. (2020). Recommender systems and their ethical challenges. *Ai & Society*, 35, 957–967.
- Narang, K., Song, Y., Schwing, A., & Sundaram, H. (2021). Fuserec: Fusing user and item homophily modeling with temporal recommender systems. *Data Mining and Knowledge Discovery*, 35, 837–862.
- Natarajan, S., Vairavasundaram, S., Natarajan, S., & Gandomi, A. H. (2020). Resolving data sparsity and cold start problem in collaborative filtering recommender system using linked open data. *Expert Systems with Applications*, 149, Article 113248. <http://dx.doi.org/10.1016/j.eswa.2020.113248>.
- Panagiotakis, C., Papadakis, H., Papagrigoriou, A., & Fragopoulou, P. (2021). Improving recommender systems via a dual training error based correction approach. *Expert Systems with Applications*, 183, Article 115386. <http://dx.doi.org/10.1016/j.eswa.2021.115386>.
- Patro, G. K., Porcaro, L., Mitchell, L., Zhang, Q., Zehlike, M., & Garg, N. (2022). *Fair ranking: A critical review, challenges, and future directions*. FAccT '22 (pp. 1929–1942). New York, NY, USA: Association for Computing Machinery, <http://dx.doi.org/10.1145/3531146.3533238>.
- Pujahari, A., & Sisodia, D. S. (2019). Modeling side information in preference relation based restricted boltzmann machine for recommender systems. *Information Sciences*, 490, 126–145. <http://dx.doi.org/10.1016/j.ins.2019.03.064>.
- Pujahari, A., & Sisodia, D. S. (2020a). Model-based collaborative filtering for recommender systems: An empirical survey. In *2020 first international conference on power, control and computing technologies* (pp. 443–447). <http://dx.doi.org/10.1109/ICPC2T48082.2020.9071454>.
- Pujahari, A., & Sisodia, D. S. (2020b). Pair-wise preference relation based probabilistic matrix factorization for collaborative filtering in recommender system. *Knowledge-Based Systems*, 196, Article 105798. <http://dx.doi.org/10.1016/j.knsys.2020.105798>.
- Pujahari, A., & Sisodia, D. S. (2022). Handling dynamic user preferences using integrated point and distribution estimations in collaborative filtering. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 52, 6639–6651. <http://dx.doi.org/10.1109/TSMC.2022.3148675>.
- Rafailidis, D., & Nanopoulos, A. (2016). Modeling users preference dynamics and side information in recommender systems. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 46, 782–792. <http://dx.doi.org/10.1109/TSMC.2015.2460691>.
- Sahoo, N., Singh, P. V., & Mukhopadhyay, T. (2012). A hidden Markov model for collaborative filtering. *Management Information Systems Quarterly*, 36, 1329–1356.
- Sun, Z., Wu, B., Wang, Y., & Ye, Y. (2022). Sequential graph collaborative filtering. *Information Sciences*, 592, 244–260.
- Wang, S., Cao, L., Wang, Y., Sheng, Q. Z., Orgun, M. A., & Lian, D. (2021). A survey on session-based recommender systems. *ACM Computing Surveys (CSUR)*, 54, 1–38.
- Wang, R., Wu, Z., Lou, J., & Jiang, Y. (2022). Attention-based dynamic user modeling and deep collaborative filtering recommendation. *Expert Systems with Applications*, 188, Article 116036. <http://dx.doi.org/10.1016/j.eswa.2021.116036>.
- Wu, L., He, X., Wang, X., Zhang, K., & Wang, M. (2022). A survey on accuracy-oriented neural recommendation: From collaborative filtering to information-rich recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 1. <http://dx.doi.org/10.1109/TKDE.2022.3145690>.
- Wu, S., Sun, F., Zhang, W., Xie, X., & Cui, B. (2022). Graph neural networks in recommender systems: a survey. *ACM Computing Surveys*, 55, 1–37.
- Yang, L., Liu, Z., Wang, Y., Wang, C., Fan, Z., & Yu, P. S. (2022). Large-scale personalized video game recommendation via social-aware contextualized graph neural network. In *WWW '22, Proceedings of the ACM Web Conference 2022* (pp. 3376–3386). New York, NY, USA: Association for Computing Machinery, <http://dx.doi.org/10.1145/3485447.3512273>.
- Zhang, Y., Bi, X., Tang, N., & Qu, A. (2021). Dynamic tensor recommender systems. *Journal of machine learning research*, 22.
- Zhang, H., Ni, W., Li, X., & Yang, Y. (2018). Modeling the heterogeneous duration of user interest in time-dependent recommendation: A hidden semi-markov approach. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 48, 177–194. <http://dx.doi.org/10.1109/TSMC.2016.2599705>.
- Zhang, M., Wu, S., Yu, X., Liu, Q., & Wang, L. (2022). Dynamic graph neural networks for sequential recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 1. <http://dx.doi.org/10.1109/TKDE.2022.3151618>.
- Zhao, Y., Wang, X., Chen, J., Wang, Y., Tang, W., He, X., & Xie, H. (2022). Time-aware path reasoning on knowledge graph for recommendation. *ACM Transactions on Information Systems*, 41, <http://dx.doi.org/10.1145/3531267>.